

Three.js Game Skills — 完整技术分析报告

项目: threejs-game-skills

版本: 0.1.0

作者: Majid Manzarpour

许可证: MIT

报告日期: 2025-07-14

目录

- 项目概览
- 整体架构
- 技能生态系统与工作流
- 各技能架构深度拆解
 - threejs-game-director (游戏总监)
 - threejs-gameplay-systems (玩法系统)
 - threejs-aaa-graphics-builder (AAA 图形构建器)
 - threejs-game-ui-designer (UI 设计器)
 - threejs-debug-profiler (调试分析器)
 - threejs-qa-release (QA 发布)
 - threejs-3d-generator (3D 生成器)
 - threejs-image-generator (图片生成器)
 - threejs-audio-generator (音频生成器)
- 工作原理解析
- 总结与评估

1. 项目概览

1.1 项目定位

threejs-game-skills 是一个为 AI 编程助手 (Codex / Claude Code) 设计的**技能编排系统**, 目标是用自然语言驱动完整的 Three.js 浏览器游戏开发流水线。

它**不是一个游戏引擎**，而是一套将游戏开发知识、最佳实践、质量门禁和外部 AI 资产生成能力封装为可复用 "技能" 的 **AI Agent 知识框架**。

1.2 核心设计哲学

整个系统的设计围绕一个核心信念：**AI 构建的游戏不能只是 "能跑"，必须 "能玩、好看、已验证"**。为此，项目构建了一个严密的质量门禁系统：

- **技能加载门禁** — 要求加载所有相关 sibling skill 文件后才开始实现
- **参考文件门禁** — 每个阶段启动前必须加载对应的 reference 文件
- **外部资产生成门禁** — 在决定跳过外部生成前强制运行凭据探测
- **视觉评分卡** — 用 10 类别 0-3 分的量化标准消除主观判断
- **报告审计器** — 自动化检查最终报告是否包含所有必需证据

1.3 技术栈默认

层面	选择	备注
语言	TypeScript	全类型安全
构建	Vite	快速 HMR
3D 引擎	Three.js	模块化导入 (<code>three/addons/ ...</code>)
物理引擎	Rapier (默认)	Rust/WASM，刚体+传感器+CCD
物理备选	cannon-es	JS-only 轻量 fallback
调试面板	lil-gui	本地调参
浏览器测试	Playwright	截图+Canvas 像素检测
包管理	npm	—

2. 整体架构

2.1 目录结构

```
threejs-game-skills.git/
├── README.md           # 用户文档
├── LICENSE             # MIT
├── package.json        # 维护者脚本 (check/validate/install)
├── install.sh          # 本地安装器
├── .gitignore
└── AGENTS.md          # Agent 全局指令 (技术栈、质量标准、技能索引)
```

scripts/	# 维护者工具
— validate-skills.sh	# 技能格式校验
— check-python-syntax.py	# Python 语法检查
skills/	# 9 个技能包（核心资产）
— threejs-game-director/	# 🎬 总监：全局编排
— threejs-gameplay-systems/	# 🎮 玩法：架构+机制+物理
— threejs-aaa-graphics-builder/	# 🌟 AAA 图形：视觉升级
— threejs-game-ui-designer/	# 🖥️ UI：界面设计
— threejs-debug-profiler/	# 🐞 调试：问题诊断+性能
— threejs-qa-release/	# ✅ QA：验证+发布
— threejs-3d-generator/	# 🏠 3D 生成：Tripo API
— threejs-image-generator/	# 🖼️ 图片生成：Gemini API
— threejs-audio-generator/	# 🔊 音频生成：ElevenLabs API

2.2 安装机制

install.sh 路由：

```
./install.sh --codex    → ~/.codex/skills/
./install.sh --claude   → ~/.claude/skills/
./install.sh --all      → 两者都安装
./install.sh --agents   → ~/.agents/skills/
./install.sh --force    → 覆盖已有同名技能
./install.sh --prune-managed → 清理旧版本
```

安装流程细节：

1. 遍历 `skills/` 下所有包含 `SKILL.md` 的目录
2. 使用 `rsync`（优先）或 `cp -R` 复制到目标 `agents skills` 目录
3. 自动排除 `node_modules/`, `dist/`, `__pycache__`, `.DS_Store` 等
4. 写入 `.threejs-game-skills-managed` 清单文件用于后续 `--prune-managed`

也可通过 `npx skills add` 远程安装：

```
npx skills add majidmanzarpour/threejs-game-skills --skill '*' -a codex -g -y
```

2.3 技能结构规范

每个技能都是**自包含**的独立包：

```
<skill-name>/
|— SKILL.md          # 技能入口 - AI 指令（必需，含 YAML frontmatter）
|— agents/           # Agent 路由配置
|   |— openai.yaml    # OpenAI 兼容的技能路由
|— references/        # 参考文件（操作模板、配方、检查清单）
|   |— *.md           # 通用参考
|   |— prompt-templates.md # 可复用 prompt 模板
```

```

├── checklists/
│   └── *.md          # 质量检查清单
├── scripts/          # 可执行脚本
│   ├── *.py | *.sh | *.mjs
├── assets/           # 静态资源
│   └── threejs-vite-game/ # 预制游戏脚手架 (仅 gameplay-systems 中)

```

SKILL.md 的 YAML frontmatter 规范:

```

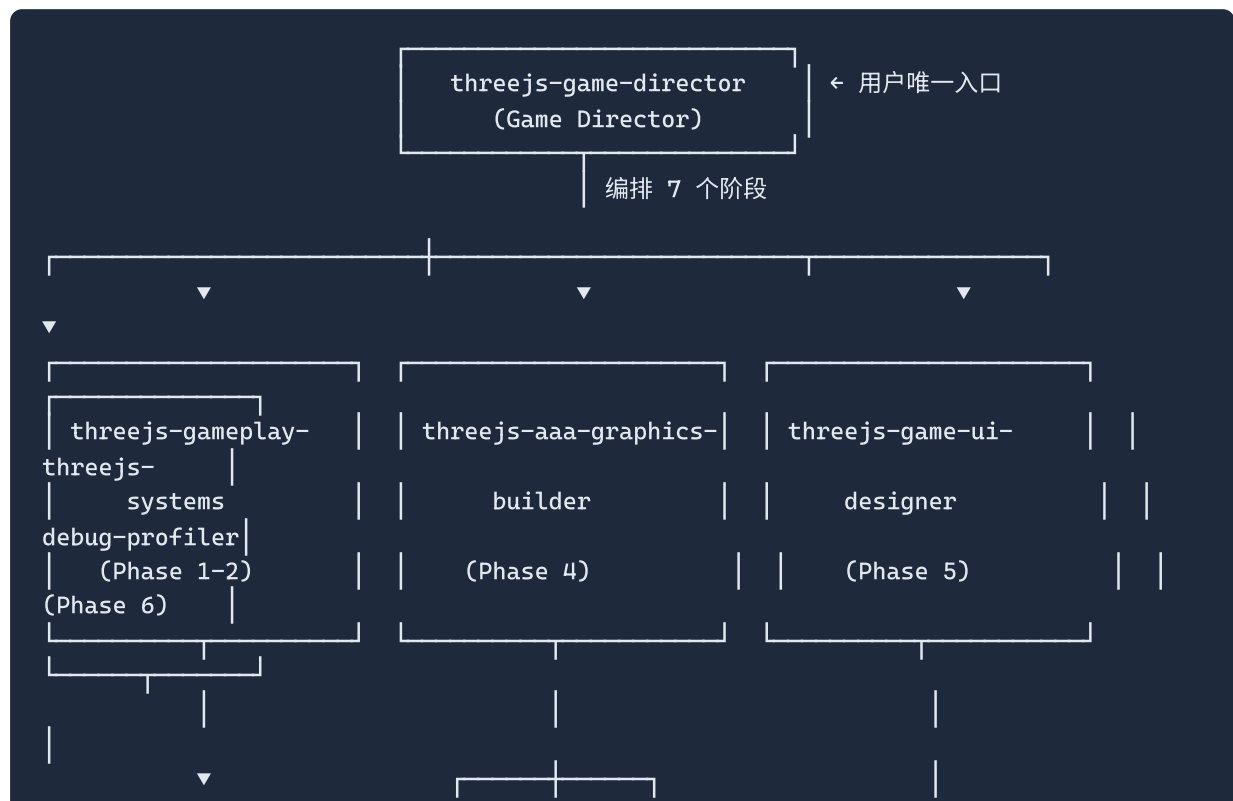
---
name: threejs-game-director
description: "简短描述 (≤120 字符), AI 阅读后决定是否调用"
---

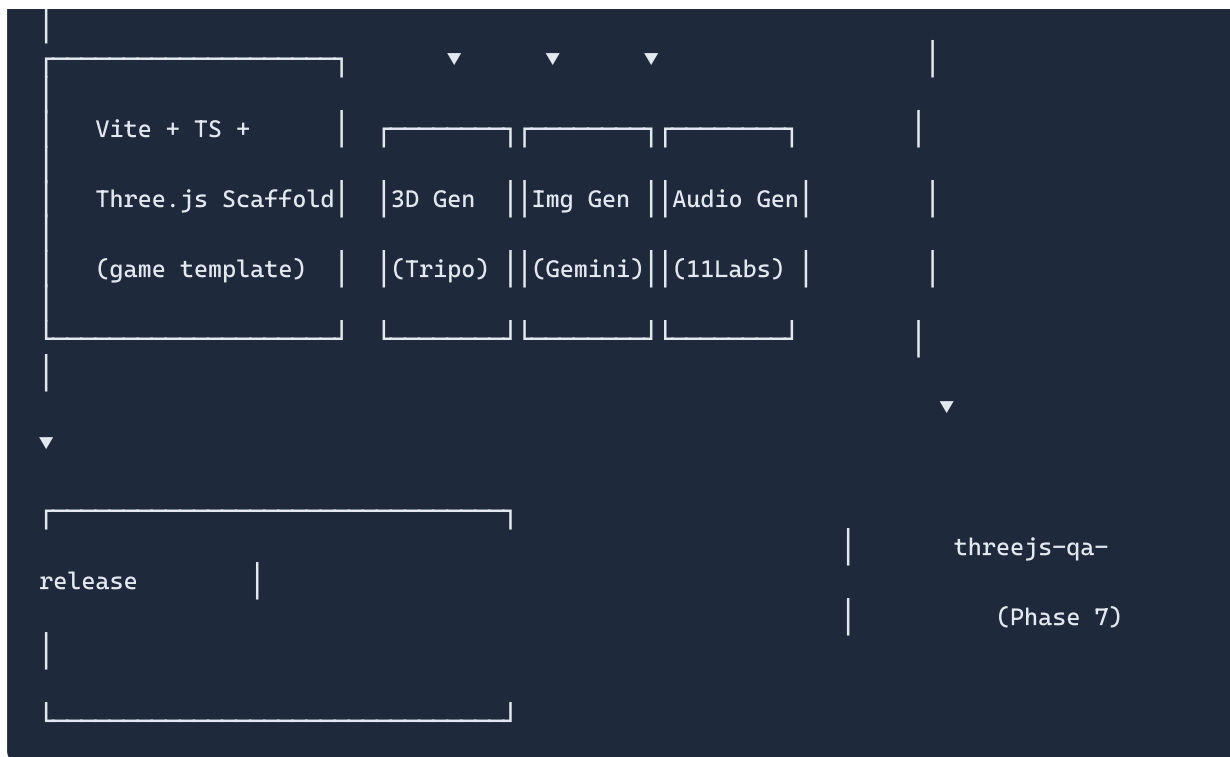
```

注意: AGENTS.md 中声明 "Prefer TypeScript, Vite, npm package imports, and Three.js modules", "Use Rapier as the default robust physics engine", 以及 "Load threejs-gameplay-systems/references/physics-engine-selection.md before adding or changing physics-heavy gameplay" 等技术栈指令对所有技能有效。

3. 技能生态系统与 workflow

3.1 技能间依赖拓扑

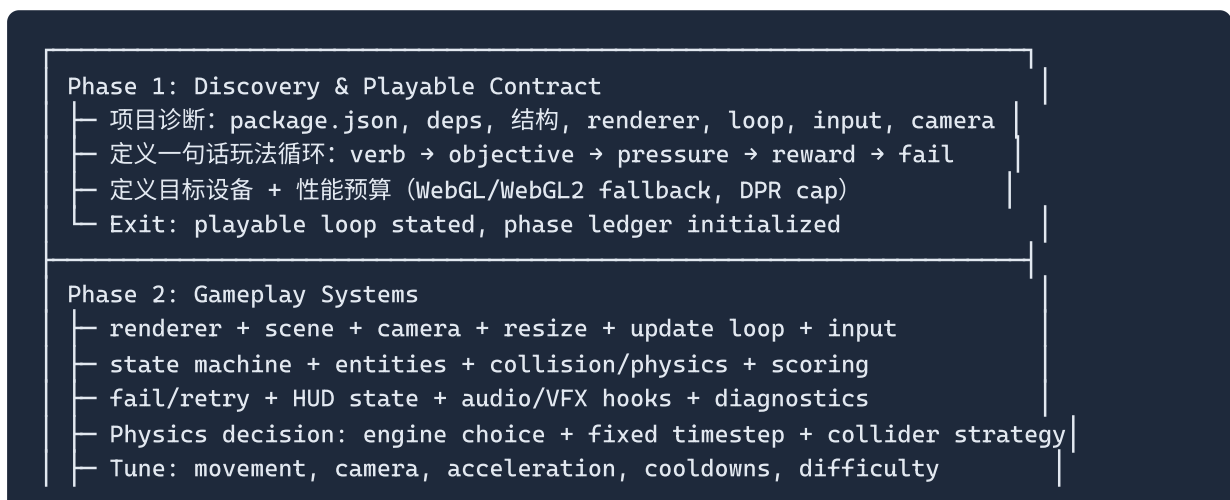




耦合关系说明:

- **Director** → **Gameplay**: 强制加载（创建/修复可玩循环）
- **Director** → **AAA Graphics**: 强制加载（视觉升级）
- **Director** → **UI Designer**: 强制加载（界面设计）
- **Director** → **Debug Profiler**: 强制加载（诊断+优化）
- **Director** → **QA Release**: 强制加载（验证+发布）
- **Director** → **3D/Image/Audio Generators**: 条件加载（premium/AAA/高保真要求时）
- **Gameplay** → **Scaffold**: 通过 `create_threejs_game.py` 创建新项目
- **AAA Graphics** → **Generators**: 高价值 surface 需要外部资产生成
- **UI Designer** → **Image Generator**: Logo/图标/GUI 艺术

3.2 完整 7 阶段开发流水线



Exit: build ☒ | browser ☒ | nonblank canvas ☒ | input path ☒

Phase 3: External Asset Sourcing

- Run probe_asset_credentials.sh → SET/MISSING output
- Load 3D/Image/Audio generator SKILL.md files
- Fill asset sourcing ledger per surface
- Generate at least 1 high-value external output for premium claims
- Exit: credential output + sourcing ledger + generated assets

Phase 4: AAA Graphics

- Score active-play screenshots (10 categories)
- Add graphics architecture: material library, model factories, VFX
- Upgrade every weak visible surface
- Add lighting/render/material polish
- Add event-driven VFX
- Re-score screenshots until every category ≥ 2
- Exit: before/after scorecard + screenshots + renderer diagnostics

Phase 5: UI

- Inventory UI states: gameplay, pause, settings, fail, win, loading
- Replace stat-card grids with authored clusters, meters, icons
- Responsive constraints + safe-area padding + touch targets
- Wire UI to game state (single source of truth)
- Exit: screenshots + text-fit + safe-area + touch-target checks

Phase 6: Debug & Profile

- Reproduce locally, read console/page/network errors
- Check canvas, renderer, camera, loop, assets, input, resize
- Performance: baseline → bottleneck → single optimization → remeasure
- Exit: root cause/bottleneck + baseline/post metrics

Phase 7: QA & Release

- Build/typecheck + dev/preview server
- Console/page/network error check
- Canvas nonblank pixel check
- Desktop + mobile active-play screenshots
- Main input → objective → fail/retry path
- Production build + static host checks + API key scan
- Exit: pass/fail + screenshots + issues + deployment notes

3.3 三层门禁系统

第一层：技能加载门禁 (Skill Loading Gate)

- 强制要求加载 5 个核心 sibling SKILL.md
- 条件加载 3 个 generator SKILL.md (premium 需求)
- 路径探试顺序: ../<skill>/ → ~/.claudeskills/ →
~/.codexskills/ → ~/.agentskills/ →
skills/<skill>/ (repo source)
- 失败则 fallback 到 director-phase-os.md
- 记录技能加载账本

↓

第二层：参考文件门禁（Reference Gate）

- 每个阶段启动前必须加载对应 `references/*.md`
- 阶段未加载所需参考 → 阶段不能标记为 `done`
- 20+ 个可能的参考文件按需加载
- 记录参考文件账本
- `thorough mode` 为默认模式 (`broad/premium` 请求)

↓

第三层：外部资产生成门禁（Asset Sourcing Gate）

- 运行 `probe_asset_credentials.sh` → SET/MISSING 输出
- "key unavailable" 不是有效跳过理由，除非探测输出为证
- 按 `surface` 决策: `procedural` / `generator` / `hybrid`
- Premium hero `surface`: `procedural-only` 不允许，除非有真实 `blocker` (探测MISSING / API错误 / 用户要求offline)
- 至少 1 个高价值 `surface` 需有外部证据 (UUID/GLB路径/图片)

4. 各技能架构深度拆解

4.1 threejs-game-director（游戏总监）— 编排核心

职责: 系统心脏。负责技能加载、阶段路由、参考文件门禁、账本管理和最终交付报告。

位置: `skills/threejs-game-director/`

4.1.1 内部结构

```
threejs-game-director/
├── SKILL.md                    # 编排核心指令 (~500行)
├── agents/openai.yaml         # OpenAI 路由
├── references/
│   └── director-phase-os.md   # Fallback 阶段操作系统 (Phase 1-7 详
解)
├── prompt-templates.md       # 可复用 prompt 模板
├── scripts/
│   ├── probe_asset_credentials.sh # 凭据探测脚本
│   └── audit_reference_report.py  # 报告审计器
```

4.1.2 关键数据结构 — 四大账本

1. 技能加载账本 (Skill-Loading Ledger)

```
Director: active
Sibling skills loaded:
```

- Gameplay systems: yes/no, path or reason:
- AAA graphics: yes/no, path or reason:
- UI: yes/no, path or reason:
- Debug/profile: yes/no, path or reason:
- QA/release: yes/no, path or reason:
- 3D generator: yes/no/not-needed, path or reason:
- Image generator: yes/no/not-needed, path or reason:
- Audio generator: yes/no/not-needed, path or reason:

2. 参考文件账本 (Reference Ledger)

Required references loaded:

- Gameplay workflows: yes/no/not-needed, path or reason:
- Physics engine selection: yes/no/not-needed, path or reason:
- Gameplay/new-game checklists: yes/no/not-needed, path or reason:
- Visual scorecard: yes/no/not-needed, path or reason:
- Graphics implementation blueprint: yes/no/not-needed, path or reason:
- Model recipes: yes/no/not-needed, path or reason:
- Render recipes: yes/no/not-needed, path or reason:
- UI patterns: yes/no/not-needed, path or reason:
- Debug/profile checklists: yes/no/not-needed, path or reason:
- QA/release checklists: yes/no/not-needed, path or reason:
- 3D generator API notes: yes/no/not-needed, path or reason:
- 3D generator Three.js integration: yes/no/not-needed, path or reason:
- Audio workflows: yes/no/not-needed, path or reason:

3. 外部资产生成账本 (External Asset Sourcing Ledger)

External asset sourcing:

- Credential probe output: TRIPO_API_KEY=SET|MISSING ...
- Hero/player source: procedural / threejs-image-generator / threejs-3d-generator / hybrid
- Enemies/vehicles/weapons source:
- Signature props/pickups source:
- World/sky/background source:
- Materials/textures/decals source:
- Logos/icons/GUI art source:
- Audio/SFX/voice source:
- External assets generated: yes/no, outputs or reason:
- Audio assets generated: yes/no/not-needed, outputs or reason:

4. 阶段执行账本 (Phase Execution Ledger)

Gameplay systems: pending/running/done/skipped - evidence:
 External asset sourcing: pending/running/done/skipped - evidence:
 AAA graphics: pending/running/done/skipped - evidence:
 UI: pending/running/done/skipped - evidence:
 Debug/profile: pending/running/done/skipped - evidence:
 QA/release: pending/running/done/skipped - evidence:

4.1.3 凭据探测脚本

`probe_asset_credentials.sh` 的核心逻辑：

```
# 用登录 shell 方式 source 用户 profile, 确保检测到交互式 profile 中的变量
zsh -lc '
    source "$HOME/.zprofile" >/dev/null 2>&1 || true
    source "$HOME/.zshrc" >/dev/null 2>&1 || true
    printf "TRIPO_API_KEY=%s\n" "${TRIPO_API_KEY:+SET}"
    printf "GEMINI_API_KEY=%s\n" "${GEMINI_API_KEY:+SET}"
    printf "ELEVENLABS_API_KEY=%s\n" "${ELEVENLABS_API_KEY:+SET}"
'
```

输出格式（永不泄露实际值）：

```
TRIPO_API_KEY=SET
GEMINI_API_KEY=MISSING
ELEVENLABS_API_KEY=SET
```

4.1.4 报告审计器

`audit_reference_report.py` 通过正则匹配检查最终报告是否包含所有必需标记：

基础必需标记（所有报告）：

- skill-loading ledger, reference ledger, phase ledger
- gameplay systems, aaa graphics, ui, debug/profile, qa/release

Premium 额外必需：

- 10 类别评分卡 + average + automatic failures
- external asset sourcing + credential probe output
- build, console, page error, desktop, mobile, screenshot, canvas, pixel
- 外部资产证据：UUID pattern / assets/models/*.glb / *.png 等

Physics 额外必需：

- physics engine, timestep, collider

Audio 额外必需：

- audio generator, elevenlabs_api_key=
- assets/audio/*.mp3 等音频文件路径

允许的跳过理由（经正则匹配验证）：

- 用户明确要求 offline / no external AI
- Credential probe 输出 MISSING
- API/network/quota error（有实际命令和错误摘要）
- 低价值重复 prop（已通过评分卡 ≥ 2 且账本解释理由）

4.1.5 Claude 兼容性规则

Claude-style skill runner 可能只调用 director 这一个 skill。

此时 director 必须：

1. 用 filesystem read 工具手动加载 sibling SKILL.md 文件
2. 加载每个阶段的 reference 文件
3. 如果某个 SKILL.md 无法加载，使用 director-phase-os.md 作为 fallback
4. 不能声称调用了未实际加载的 skill

4.2 threejs-gameplay-systems（玩法系统） — 游戏骨架

职责: 构建可玩的游戏循环。从项目脚手架创建到架构设计、输入系统、碰撞/物理、计分系统和手感调校。

位置: skills/threejs-gameplay-systems/

4.2.1 内部结构

```
threejs-gameplay-systems/
├── SKILL.md
├── agents/openai.yaml
├── scripts/
│   └── create_threejs_game.py          # 脚手架创建脚本
├── assets/
│   └── threejs-vite-game/              # 预制 Vite+TS+Three.js 游戏模板
│       ├── package.json
│       ├── package-lock.json
│       ├── tsconfig.json
│       ├── vite.config.ts
│       ├── playwright.config.ts
│       ├── index.html
│       └── src/
│           ├── main.ts                  # DOM 引导 + HMR
│           ├── styles.css               # 响应式布局 + 触控 CSS
│           └── core/
│               ├── Loop.ts              # 单一 rAF 循环
│               ├── Renderer.ts          # WebGLRenderer 工厂 + resize
│               └── InputController.ts    # 键盘 + 虚拟摇杆 + 冲刺按钮
│           ├── entities/
│               ├── Player.ts            # 玩家实体（胶囊体 + 锥形鼻）
│               └── Pickup.ts            # 收集物实体
│           ├── game/
│               └── Game.ts               # 主编排器
│           ├── systems/
│               ├── CameraRig.ts         # 第三人称跟随相机
│               ├── CollisionSystem.ts    # 距离检测
│               ├── Hud.ts               # DOM-based HUD
│               ├── AudioSystem.ts       # Web Audio 钩子
│               └── DebugTools.ts        # lil-gui 调试面板
│           └── utils/
│               └── dispose.ts            # Three.js 资源释放
└── tests/
```

```

├── visual.spec.ts          # Playwright 视觉测试
├── scripts/
│   └── inspect-threejs-canvas.mjs # Canvas 检测
├── references/
│   ├── gameplay-workflows.md      # 玩法工作流参考
│   ├── physics-engine-selection.md # 物理引擎选择指南
│   ├── prompt-templates.md
│   └── checklists/
│       ├── new-game-definition-of-done.md # 新游戏完成定义
│       └── endless-runner-premium-quality.md

```

4.2.2 First Playable Slice 方法论

第一步：定义一句话玩法循环

verb → objective → pressure → reward → fail/retry

示例: "Move player to collect relays, under time pressure, score points, fail if hit by hazards, restart to try again"

第二步：仅实现该循环所需的机制

1. renderer + scene
2. camera + resize
3. update/render loop (Loop.ts: 单一 rAF, delta clamped to 0.05s)
4. input intents (InputController: keyboard + touch virtual stick)
5. player entity (Player: CapsuleGeometry body + ConeGeometry nose)
6. one obstacle/enemy
7. one reward/progress path (Pickup: RingGeometry + collect animation)
8. collision/trigger checks (CollisionSystem: distance-based)
9. score/status state (Game.score)
10. fail/retry state (Game.complete flag)
11. minimal HUD state (Hud: score/target/timer/status)
12. one audio/VFX hook (AudioSystem.pickup() / Hud.flashPickup())

第三步：添加诊断

```

window.__THREE_GAME_DIAGNOSTICS__ = {
  frame, elapsed, score, targetScore, complete,
  player: { position, speed },
  renderer: { calls, triangles, geometries, textures },
  canvas: { clientWidth, clientHeight, width, height, dpr }
}

```

拒绝条件: Slice 不可被控制或无法重新开始 → 不算完成

4.2.3 架构边界

```

src/
├── main          → DOM bootstrap, app lifecycle, CSS imports
├── core/         → Renderer, Loop, Resize, Input, Diagnostics
├── game/         → Orchestration, state transitions, update order, scoring
├── entities/     → Player, Enemies, Pickups, Projectiles, Obstacles
├── systems/      → Camera, Collision/Physics, Spawning, Animation, Audio, UI
└── Bridge

```

- └─ assets/ → Material libraries, procedural textures, model factories, loaders
- └─ ui/ → (当 UI 复杂度增长时从 systems 中分离)
- └─ tests/ → Browser, visual, interaction, mobile, performance smoke

4.2.4 显式更新顺序

input intents → fixed physics (if any) → gameplay systems
 → animation/VFX → camera → UI bridge → render

4.2.5 物理引擎选择阶梯

- | |
|---|
| 1. Custom collision
→ arcade triggers, pickups, lanes, runners, simple bullets |
| 2. Rapier (@dimforge/rapier3d-compat) ← 默认推荐
→ rigid bodies, sensors, CCD, balls, ramps, platforms
→ Rust/WASM, typed, fast, official JS bindings |
| 3. cannon-es ← JS-only 轻量 fallback
→ small rigid-body scenes, avoiding WASM matters |
| 4. Jolt
→ advanced experiments, more integration complexity |
| 5. Ammo.js/Bullet ← 仅已依赖项目 |
| 6. Matter.js ← 2D only |

Rapier 标准集成模式:

```
// 初始化 (一次)
import RAPIER from '@dimforge/rapier3d-compat';
await RAPIER.init();
const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });

// 固定时间步累加器
const fixedDt = 1 / 60;
let accumulator = 0;

function update(deltaSeconds: number) {
  accumulator += Math.min(deltaSeconds, 0.1); // 钳制 spike
  while (accumulator ≥ fixedDt) {
    world.timestep = fixedDt;
    world.step();
    accumulator -= fixedDt;
  }
}
```

```

// 创建动态刚体 + 球碰撞器
const body = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic()
    .setTranslation(0, 2, 0)
    .setLinearDamping(0.25)
);
world.createCollider(
  RAPIER.ColliderDesc.ball(0.5)
    .setRestitution(0.6)
    .setFriction(0.4),
  body
);

// 同步到 Three.js (单一系统中)
const t = body.translation();
const r = body.rotation();
mesh.position.set(t.x, t.y, t.z);
mesh.quaternion.set(r.x, r.y, r.z, r.w);

// CCD 用于高速物体
RAPIER.RigidBodyDesc.dynamic().setCcdEnabled(true);

// 传感器/触发器
RAPIER.ColliderDesc.ball(1)
  .setSensor(true)
  .setActiveEvents(RAPIER.ActiveEvents.COLLISION_EVENTS);

```

4.2.6 游戏手感调校维度

维度	说明
移动速度 + 加速度	<code>tuning.speed = 5.8</code> , <code>tuning.acceleration = 13</code>
相机距离/跟随/预瞄	<code>cameraLag = 0.16</code> , <code>offset = (0, 9.5, 9.5)</code>
反应窗口 + 障碍间距	关卡设计参数
跳跃/加速/攻击冷却	<code>dashMultiplier = 1.75</code>
收集物磁力 + 奖励时机	距离阈值可调
命中反馈 + 重启速度	动画/音频/VFX
难度爬升 + 节奏	动态调整

使用 `lil-gui` 进行实时调参:

```

// DebugTools 暴露 tuning 对象供 lil-gui 面板调节
this.debugTools = new DebugTools(this.tuning, () => {
  this.renderer.toneMappingExposure = this.tuning.exposure;
});

```

```
resizeRenderer(this.renderer, this.camera, this.tuning.maxDpr);
});
```

4.2.7 脚手架创建脚本

`create_threejs_game.py` 的工作流：

1. 验证源脚手架存在
2. 检查目标目录（为空或 `--force`）
3. `shutil.copytree` 复制整个 `threejs-vite-game/` →
排除 `node_modules`, `dist`, `artifacts`, `__pycache__`
4. 重写 `package.json` 和 `package-lock.json` 的 `name` 字段
5. 输出: `"Next: cd <target> && npm install && npm run dev"`

4.3 threejs-aaa-graphics-builder（AAA 图形构建器）— 视觉升级

职责: 将原型级视觉效果升级到 premium/AAA 级别。系统内最大的技能包。

位置: `skills/threejs-aaa-graphics-builder/`

4.3.1 内部结构

```
threejs-aaa-graphics-builder/
├── SKILL.md
├── references/
│   ├── visual-scorecard.md           # 10 类别视觉评分卡
│   ├── implementation-blueprint.md   # 图形架构蓝图
│   ├── model-recipes.md              # 程序化模型配方
│   ├── render-recipes.md             # 渲染/光照/VFX 配方
│   └── prompt-templates.md
├── checklists/
│   ├── aaa-game-quality-gate.md      # AAA 质量门禁
│   ├── aaa-visual-scorecard.md
│   ├── procedural-model-quality.md
│   ├── material-lighting-quality.md
│   └── performance-safe-visual-detail.md
```

4.3.2 10 类别视觉评分卡

评分标准：

- 0: Placeholder – 默认几何体，稀疏世界，不可读状态
- 1: Basic styled – 可玩，有主题，但仍是原型资产
- 2: Premium stylized – 作者化轮廓，材质系统，可读状态
- 3: Showcase – 强艺术方向，难忘的英雄和世界，密集细节

#	类别	0 分	1 分	2 分 (Premium)	3 分 (Showcase)
1	Art direction	无主题	主题=颜色/雾	主题影响形态+材质+UI+反馈	每面独特风格
2	Hero/player	几何体堆叠	基础物体+发光	作者化轮廓+贴花+状态提示	分层构造+表现力反馈
3	Obstacles/enemies	方块/锥体/球	重复+换色	3种变体+预警+材质提示	多样化家族+动画+清晰度
4	Rewards/interactables	球/环/token	重复+简单发光	2种形态+收集状态+UI反馈	期望感+动画+运动中清晰
5	World/environment	平面/空场/盒子	主题化但稀疏	分层道具套件+前后景+尺度	密集作者化世界+可读性
6	Materials/textures	纯色	基础 roughness	共享材质角色+贴花+饰线	丰富材质语言+测量纹理
7	Lighting/render	默认光	雾/辉光主导	有意的色调映射+主补轮廓光	电影级但可读+严格后处理
8	VFX/motion	无/随机	通用粒子/轨迹	事件驱动(加速/拾取/命中)	高影响力+澄清玩法+高性能
9	UI/HUD	调试文字/缺失	通用统计卡	类型特定+计量/图标+响应式	强层级+流畅过渡+内聚
10	Performance evidence	无指标	口头"还行"	渲染器计数+截图	基准/优化前后+预算

Premium 阈值: 每类 ≥ 2 , 平均 ≥ 2.3 , 需活跃桌面+移动截图, 渲染器诊断

Showcase 阈值: ≥ 6 类得分 3, 平均 ≥ 2.7 , 含性能优化前后对比

8 项自动失败条件:

1. 活跃截图为几何体主导
2. 世界是拉伸盒子/平面/稀疏空场
3. Hero 资产=默认几何体+发光
4. 障碍物或奖励=一个重复轮廓
5. HUD=矩形统计/调试卡片
6. 雾/暗/辉光/粒子 掩盖缺失几何
7. UI 遮挡游戏路径/文字截断/移动端安全区失败
8. 无活跃游戏截图 / 无渲染器诊断

4.3.3 图形架构蓝图

推荐的模块结构:

```
src/assets/
├── MaterialLibrary.ts           # 命名材质角色
├── ProceduralTextures.ts       # Canvas 程序化纹理
├── DecalShapes.ts              # 贴花形状
├── ModelDiagnostics.ts         # 模型诊断
├── ImportedAssetRegistry.ts    # 外部导入模型注册
├── modelFactories/
│   ├── HeroFactory.ts          # 英雄/玩家工厂
│   ├── ObstacleFactory.ts      # 障碍物工厂
│   ├── RewardFactory.ts        # 奖励物工厂
│   └── WorldPropKit.ts         # 世界道具套件
└──
src/systems/
├── LightingRig.ts              # 灯光栈
├── RenderPipeline.ts           # 渲染管线
├── VfxSystem.ts                # VFX 系统
├── WorldArtDirector.ts         # 世界艺术总监
└── QualityDiagnostics.ts       # 质量诊断
```

材质角色库 (Material Library):

角色	用途
bodyPrimary	玩家/世界主体外壳
bodySecondary	面板对比
trim	轨道、倒角高亮、边框
hazard	危险面、伤害提示
reward	收集物面
glass	驾驶舱、护盾、镜头
emissiveSignal	发光条、状态灯
groundContact	暗磨砂阴影接收面
decalDark / decalLight	面板线、划痕、数字、图标

4.3.4 世界分层模型 (World Art Director)

Motion Layer: speed lines, particles, trail strips, screen-space	← 最前层
Play Layer: ground, lanes, rails, objective path, hazards	← 核心可玩层
Near Layer: speed props, signs, arches, barriers, foreground occluders	← 速度感+尺度

Mid Layer: buildings, cliffs, hangars, pillars, platforms, machinery	← 定义走廊
Far Layer: skyline, terrain silhouettes, cloud/fog cards, parallax planes	← 深度+氛围

4.3.5 模型配方 (model-recipes.md)

最低 Premium 资产通行证:

- 1 个 hero/player 模型（可读的前/上/侧 + 3 种状态提示）
- 3 个障碍/敌人变体（独特轮廓 + 预警）
- 2 个奖励/交互变体（idle + collect 状态）
- 1 个世界道具套件（≥ 8 个可复用部件）
- 1 个材质套件（trim + 贴花 + 面板线 + 发光遮罩）
- 碰撞代理 + 渲染器诊断

英雄载具配方:

Core hull	→ ExtrudeGeometry 或自定义 BufferGeometry
Nose/front	→ 楔形、进气口、传感器带、保险杠
Cockpit	→ 球/Lathe 分段玻璃罩
Engines	→ 圆柱/锥/管 + 喷嘴环 + 发光盘 + 热鳍片
Wings/fins	→ 挤出三角/曲面板 + 倒角/trim
Undercarriage	→ 滑橇、起落架、轨道夹具、悬挂臂
Decals	→ 面板线、数字标记、派系符号、危险标
State cues	→ 加速火焰、护盾壳、伤害焦痕
Collision	→ 一个胶囊/盒子/球组

拒绝条件: Hero 主要是盒子+两个圆柱+发光

程序化几何技术选择:

技术	用途
ExtrudeGeometry	面板、鳍、翼、徽章、标志
LatheGeometry	胶囊、穹顶、引擎、管道、炮塔底座
TubeGeometry	电缆、轨道、轨迹、导管
Custom BufferGeometry	锥形外壳、岩石、碎片、楔形
ShapeGeometry	贴花、平图标、trim 条
InstancedMesh	窗户、螺栓、车道标、碎片、草

技术	用途
LOD	背景变体、密集道具降级

4.3.6 渲染配方 (render-recipes.md)

灯光栈:

Key light	→ 定义形态和方向
Fill light	→ 保持可玩对象可读
Rim/back	→ 分离玩家和障碍与背景
Practical	→ 信标、引擎、拾取物、竞技场标记
Contact	→ 接触阴影或阴影 blob

后处理纪律:

- Bloom: 仅作者化的发光元素
- Vignette: 微妙，不过暗
- Film grain: 低透明度
- Chromatic aberration: 仅在事件驱动或极微妙时
- Motion blur: 偏好几何轨迹或粒子

事件驱动 VFX:

事件	效果
Boost	引擎锥、轨迹带、FOV 缓动、侧条纹
Pickup	环收缩、碎片爆发、分数轨迹、HUD 脉冲
Hit	冲击环、碎片、伤害闪烁、短暂暂停
Near miss	边缘火花、徽章脉冲
Shield	折射壳、边缘脉冲、吸收波纹
Spawn	预期脉冲、预警、溶解缩放

4.4 threejs-game-ui-designer (UI 设计器) — 界面工艺

职责: 使 UI 成为有意的、可读的、响应式的、类型特定的界面系统。

位置: `skills/threejs-game-ui-designer/`

4.4.1 内部结构

```

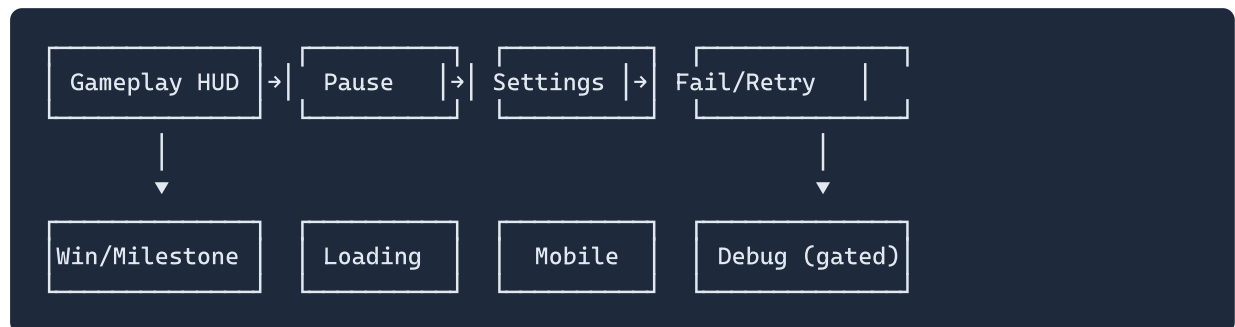
threejs-game-ui-designer/
├── SKILL.md
├── references/
│   ├── ui-patterns.md           # UI 设计模式
│   ├── prompt-templates.md
│   └── checklists/
│       ├── game-ui-quality.md
│       ├── hud-readability.md
│       ├── responsive-ui-fit.md
│       └── mobile-input.md

```

4.4.2 UI 核心原则

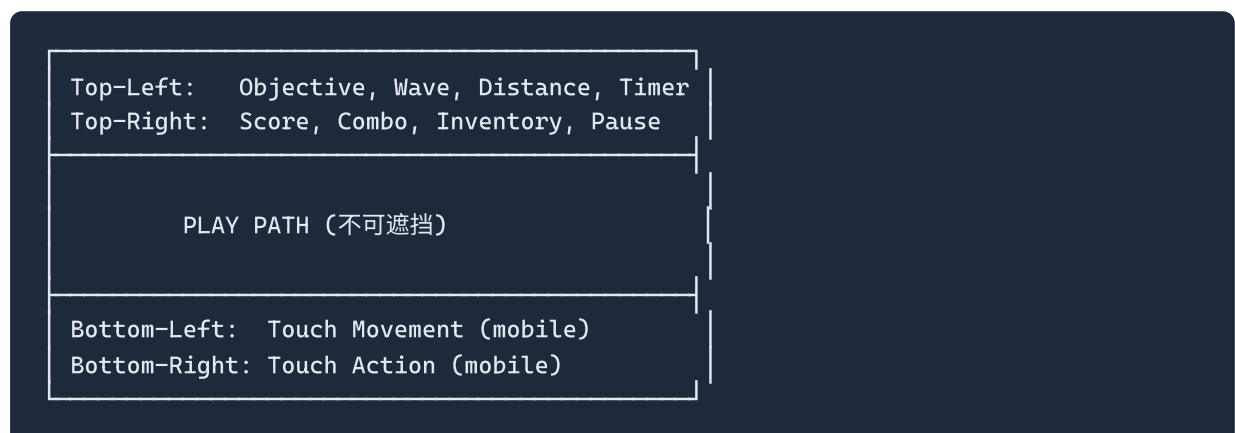
1. **构建游戏界面，不是 web dashboard**
2. **优先级层级**: 生存/状态 > 目标/进度 > 即时反馈 > 氛围
3. **使用**: meters, icons, reticles, badges, alerts, cooldown rings, minimaps, diegetic labels → **而非 generic stat cards**
4. **UI 永远不能遮挡**: 玩家、威胁、收集物、下一个决策
5. **UI 反映世界艺术方向**: 材质提示、颜色角色、图标形状、运动语言
6. **不用文字解释显而易见的操作** — 用图标、可供性或直接交互

4.4.3 8 个必需 UI 状态



Premium 游戏不允许只有一种 HUD 状态

4.4.4 HUD 组合区域



Center-Top: Event banners, Combo, Warnings
Near-World: Diegetic labels, Target markers

4.4.5 响应式约束规则

```
/* ✓ 使用稳定尺寸 */  
.score { width: 4ch; } /* 固定宽度数字槽 */  
.hud-text { font-size: clamp(12px, 2vw, 18px); } /* CSS clamp */  
  
/* ✗ 避免 */  
.hud-text { font-size: 2vw; } /* 纯 viewport 缩放 */  
.hud-text { letter-spacing: -0.05em; } /* 负字间距 */
```

检查: Desktop (1920+), Laptop (1366-1440), Tablet (768-1024), Phone (320-428)

4.4.6 触控控制模式

```
// 关键事件处理  
this.stick.addEventListener('pointerdown', this.onStickDown);  
this.stick.addEventListener('pointermove', this.onStickMove);  
this.stick.addEventListener('pointerup', this.onStickUp);  
this.stick.addEventListener('pointercancel', this.onStickUp); // ← 防止卡住  
this.dashButton.addEventListener('pointerleave', this.onDashUp); // ← 防止卡住
```

规则:

- 触控目标 ≥ 44 CSS px
- 使用 `touch-action` 仅在控制区域防止页面滚动
- `pointerup/cancel/blur/leave` 全部处理, 防止控件卡住
- 安全区使用 `env(safe-area-inset-*)`

4.4.7 状态绑定规则

```
UI 读取 ← Game State (单一数据源)  
UI 事件 → Game Intents (不直接修改模拟内部)  
UI 必须在以下事件更新: pause, restart, resize, orientation,  
                           mute, fail/win, score, health, boost,  
                           combo, inventory, accessibility
```

4.5 threejs-debug-profiler (调试分析器) — 问题诊断+性能

职责: 从根因诊断到性能优化, 覆盖渲染、运行时、加载、动画、物理、输入、移动端所有问题域。

位置: `skills/threejs-debug-profiler/`

4.5.1 内部结构

```
threejs-debug-profiler/  
├── SKILL.md  
├── references/  
│   ├── debug-profile-checklists.md      # 综合调试/性能清单  
│   ├── prompt-templates.md  
│   └── checklists/  
│       ├── scene-debugging.md  
│       ├── performance-profile.md  
│       └── mobile-input.md
```

4.5.2 调试分流顺序

1. Reproduce locally (same command/URL)
2. Capture console + page + network errors
3. Confirm correct build is served (not another app on same port)
4. Classify owner: renderer | scene | camera | loop | assets |
audio | input | physics | UI | CSS | build | perf
5. Fix root cause in owning module
6. Retest exact broken path

4.5.3 空白/错误画布 —— 13 步诊断序列

1. Canvas 存在于 DOM?
2. Canvas CSS 尺寸非零且可见?
3. Drawing buffer 尺寸匹配 DPR?
4. WebGL context 创建成功?
5. 只有一个活动循环在渲染?
6. Camera: aspect, projection, near/far, 指向可见内容?
7. Scene 有可见物体在预期位置?
8. Materials: opacity, transparent, side, depth, colorSpace, fog?
9. Lights 存在 (如果材质需要)?
10. Background/fog 颜色是否与物体颜色相同?
11. Resize 更新了 renderer + camera + composer + CSS?
12. CSS overlay 覆盖了 canvas?
13. Render target / composer 输出正确显示?

4.5.4 性能分析序列

1. Establish scenario: viewport, DPR, route, gameplay state, device
2. Baseline snapshot:
 - ├── FPS / frame time
 - ├── Renderer calls, triangles
 - ├── Geometries, materials, textures
 - ├── Render targets / post passes
 - └── JS heap / memory estimate

- Bundle + large assets
 - Imported model: file size, clips, texture dims
 - Physics: body count, collider count, sensors, CCD bodies, contacts
3. Classify bottleneck:
- CPU: simulation, allocations, pathfinding, physics, UI layout
 - GPU draw: draw calls, material switches, unique meshes
 - GPU fragment: overdraw, post-processing, high DPR
 - GPU vertex: high triangles, dense shadows
 - Memory: textures, render targets, undisposed resources
 - Network: large deps or assets
4. Apply ONE optimization
5. Re-measure same scenario
6. Check visual/playability regression

4.5.5 优化优先级

1. InstancedMesh for repeated detail
2. Shared geometries / materials / textures
3. Object pools for effects, bullets, pickups
4. Frustum / distance culling
5. LOD for background props
6. DPR cap or adaptive quality
7. Cheaper shadows: fewer casters, smaller maps
8. Limited post-processing passes
9. Texture atlases, compression, reuse, mipmaps
10. Avoid per-frame allocations
11. Physics: simple colliders, sleeping, fewer dynamic bodies
12. Dispose everything

4.5.6 渲染器诊断代码

```
window.__THREE_GAME_DIAGNOSTICS__ = {
  renderer: renderer.info, // { render: {calls, triangles, points, lines},
                           // memory: {geometries, textures} }

  get state() {
    return game.getDebugState(); // player, score, entities, input, etc.
  },
};

// Physics-heavy games 额外:
physics: {
  engine: 'rapier',
  timestep: 1 / 60,
  bodies: physicsWorld.bodyCount(),
  colliders: physicsWorld.colliderCount(),
  sensors,
  ccdBodies,
}
```

4.5.7 常见错误

- 不重现就猜测
- 优化 dev-server 性能而不是 production preview
- 在检查 DPR/post/shadows/instancing/culling 前就移除视觉细节
- 修复 CSS 症状而 renderer/camera 尺寸才是根因
- 添加移动控件时不测试 pointer cancel 和安全区
- 因为 canvas 非空白就忽略 console/page 错误

4.6 threejs-qa-release (QA 发布) — 质量保障

职责: 证明游戏在玩家遇到的条件下正常运作。

位置: `skills/threejs-qa-release/`

4.6.1 内部结构

```
threejs-qa-release/  
├── SKILL.md  
├── agents/openai.yaml  
├── scripts/  
│   └── inspect-threejs-canvas.mjs          # Playwright Canvas 检测  
├── references/  
│   ├── qa-release-checklists.md  
│   ├── prompt-templates.md  
│   └── checklists/  
│       ├── visual-verification.md  
│       ├── playtest-qa.md  
│       └── release.md
```

4.6.2 Browser QA Matrix

1. npm install → 依赖就绪
2. npm run build → build 通过
3. npm run dev / preview → server 启动在正确 URL
4. Browser console / page / network errors → 捕获
5. Canvas nonblank + pixel sampling → 有变化 ≠ 空白
6. Desktop active-play screenshot → 截取
7. Mobile active-play screenshot (if in scope) → 截取
8. Main input → game state changes → 验证
9. Objective/progress path → 验证
10. Fail/retry or pause/resume → 验证
11. Recent risky code paths → 触发测试
12. Physics: engine, timestep, body/collider count, CCD, restart cleanup
13. HUD: text fit, overlap, safe areas, touch targets
14. Renderer diagnostics (when graphics changed)

- 15. Imported asset paths, sizes, runtime load behavior
- 16. Audio: unlock, decode, loop cleanup, mute/volume, SFX triggers

4.6.3 交互 QA — 玩家实际做什么

Start → Move/Aim/Steer → Collect/Score → Avoid Hazard
→ Trigger State Change (combo/wave/checkpoint/shield/fail/win)
→ Pause/Resume → Restart after Fail → Resize/Rotate (mobile)

4.6.4 Canvas 检测器

`inspect-threejs-canvas.mjs` 使用 Playwright:

```
# 桌面检测
node scripts/inspect-threejs-canvas.mjs --url http://127.0.0.1:5188

# 移动端模拟
node scripts/inspect-threejs-canvas.mjs --url http://127.0.0.1:5188 --mobile

# 带截图
node scripts/inspect-threejs-canvas.mjs --url http://127.0.0.1:5188 --screenshot
```

检查项: Canvas DOM 存在、CSS 尺寸、Drawing buffer、像素采样非空白

4.6.5 Release Checks

```
✓ Production build passes
✓ Production preview / static server tested
✓ Vite base + asset URLs match target host
✓ Debug GUI + diagnostics + verbose logs GATED from player release
✓ Bundle + large assets reviewed
✓ API keys NOT in client code / checked-in files / built assets
✓ Public assets load under static hosting assumptions
✓ Browser support assumptions documented
✓ Deployment command / static artifact location reported
✓ Residual risks listed
```

4.6.6 常见 Release 失败

- 测试 dev server, 但 shipped 的是未测试的 production build
- Static host base path 破坏 assets
- Debug UI 对玩家可见
- Mobile UI 截图通过但控件不工作
- Canvas 非空白但是另一个 app 跑在该端口
- 截图是标题/idle 视角而非活跃游戏

- Premium 声明无评分卡/无渲染器诊断
- API key 或生成的临时 URL 意外暴露在客户端代码中

4.7 threejs-3d-generator (3D 生成器) — Tripo API 集成

职责: 通过 Tripo API 创建面向生产的 3D 资产。

位置: skills/threejs-3d-generator/

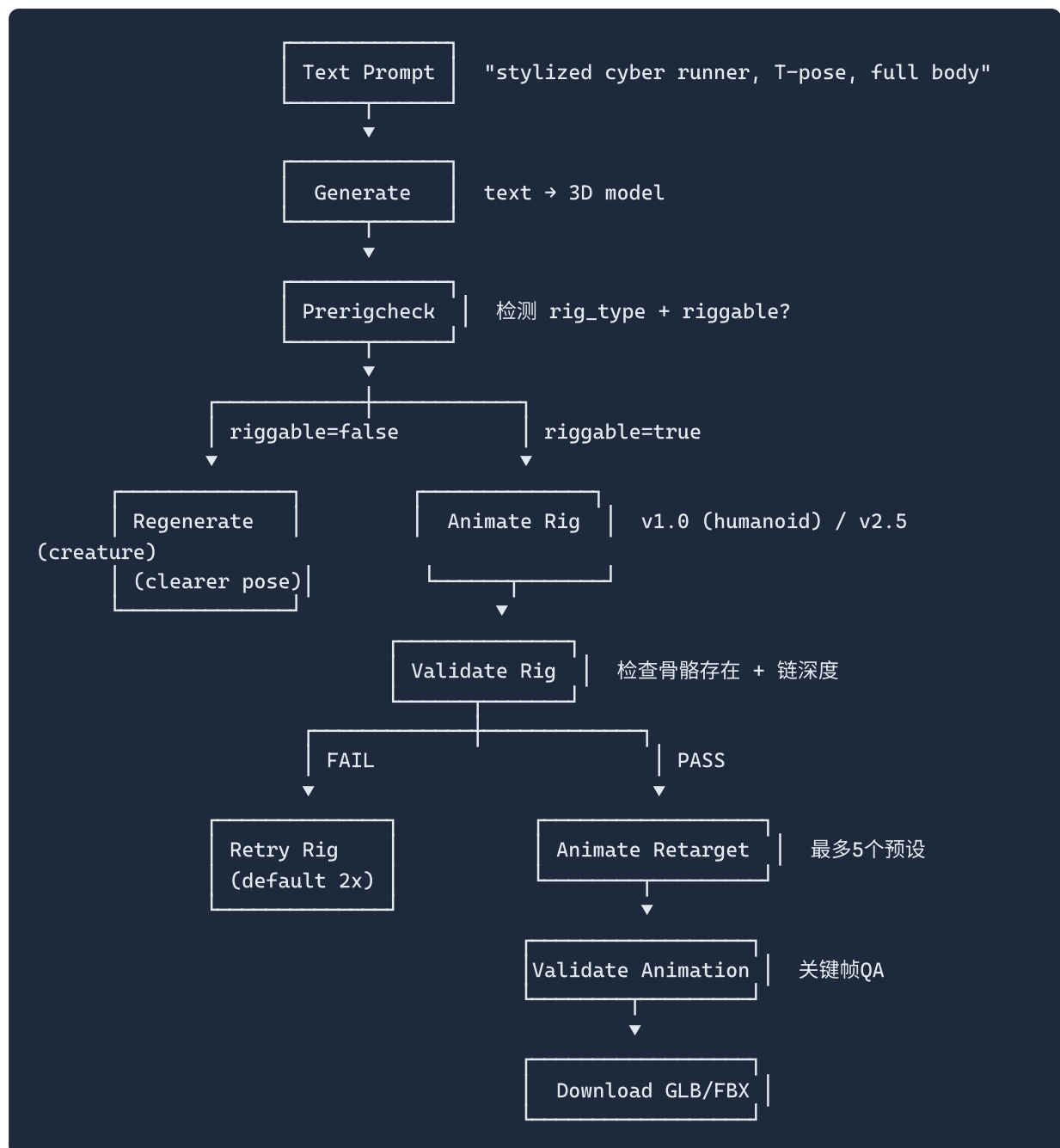
4.7.1 内部结构

```
threejs-3d-generator/  
├── SKILL.md  
├── scripts/  
│   └── threejs_3d_asset.py           # Tripo API 客户端工具  
├── references/  
│   ├── api-notes.md                 # API 参数表 + 模型版本  
│   ├── threejs-integration.md       # Three.js 导入指南  
│   └── image-generator-workflows.md  # 与 image-generator 协作
```

4.7.2 生成能力矩阵

操作	命令	说明
Text → 3D	<code>text --prompt "..." --wait --download</code>	文本描述生成 3D 模型
Image → 3D	<code>image --image concept.png --wait --download</code>	图片转 3D
Texture model	<code>postprocess --type texture_model --texture-prompt "..."</code>	重新纹理
Pre-rig check	<code>postprocess --type animate_prerigcheck</code>	验证可绑定性
Rig	<code>postprocess --type animate_rig --rig-type biped</code>	创建骨骼
Retarget animation	<code>postprocess --type animate_retargget --animations preset:idle, ...</code>	套用动画预设
Conversion	<code>postprocess --type conversion --format GLTF --face-limit 20000</code>	格式转换+减面
Stylize	<code>postprocess --type stylize_model --style voxel</code>	风格化 (voxel/LEGO/low-poly)
Character pipeline	<code>character-pipeline --prompt "..." --animations ...</code>	全自动角色管线

4.7.3 全自动角色管线 (character-pipeline)



4.7.4 关键可靠性规则（实测总结）

规则	详情
人形骨骼版本	v1.0-20240301 — v2.x limb-chain 对人形 0/16 成功率
生物骨骼版本	v2.5-20260210 — v2.x 对四足/鸟类 5-6 骨链对称性好
禁止 --animate-in-place	验证证实破坏骨骼: v1.0 镜像肢体, v2.5 爆裂蒙皮
v1.0 重定向格式	必须 FBX — GLB bake 扭骨变换空间错误, 肢体塌陷

规则	详情
v2.5 重定向格式	GLB 可用
Batch 上限	一次 <code>animate_retarget</code> 最多 5 个预设
生成姿态	武器不与手融合, T-pose/A-pose 身体对称, 全四肢可见
生物姿态	生成姿态决定动画读取方式, 需匹配预设期望
立即下载	生成的下载 URL 很快过期
重试策略	装甲/硬表面角色需最多重试, 有机网格通常一次成功
v1.0 RIG_TASK_ID	<code>animate_retarget</code> 用 RIG 任务 ID (非生成 ID)
spec 选择	Tripo 预设用 <code>--spec tripo</code> (默认); Mixamo 为外部管线专用

4.7.5 Three.js 集成规范

```
// 加载
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
const loader = new GLTFLoader();
const gltf = await loader.loadAsync('assets/models/hero.glb');

// 缩放+偏移
gltf.scene.scale.set(1, 1, 1);
// 检查 bounds, pivot, orientation

// 动画
const mixer = new THREE.AnimationMixer(gltf.scene);
const clip = THREE.AnimationClip.findByName(gltf.animations, 'NlaTrack');
const action = mixer.clipAction(clip);
action.play();

// 在 update 中
mixer.update(deltaSeconds);

// 碰撞代理 (独立于视觉网格)
const collisionProxy = new THREE.Mesh(
  new THREE.CapsuleGeometry(0.5, 1.5),
  collisionMaterial
);
// 同步位置从物理或实体
```

4.8 threejs-image-generator（图片生成器） — Gemini API 集成

职责: 为 Three.js 游戏创建 2D 资产和参考图。

位置: `skills/threejs-image-generator/`

4.8.1 内部结构

```
threejs-image-generator/  
├── SKILL.md  
├── scripts/  
│   └── generate_image.py  
# Gemini API 图片生成
```

4.8.2 生成能力

类别	用途
Concept sheets	角色概念、车辆参考
3D generation inputs	T-pose/A-pose 全身参考图（喂给 3D generator）
Texture references	地形、岩石、金属、路面、贴花
Environment images	天空、背景、地平线、星云
UI art	Logo、图标、派系标记、物品卡片、标题艺术
Image editing	已有图片的风格变体、调色板对齐、清理

4.8.3 分辨率策略

```
1K → quick concepts, icons, draft sheets  
2K → default production: image-to-3D, textures, backgrounds, UI panels  
4K → hero splash/title art, high-detail texture references
```

4.8.4 Prompt 模式

Image-to-3D 参考:

```
Create a clean 3D-generation reference image of [asset].  
Centered single object, full object visible, plain light background,  
readable silhouette, clear material zones, game-ready [genre/style],  
no motion blur, no cropped parts, no text.
```

可绑定角色参考:

```
Create a full-body [T-pose/A-pose/side-view creature] reference for 3D rigging:  
[details]. Symmetric stance, visible hands/feet/limbs, plain background,  
readable costume/anatomy layers, no weapon fused to hands.
```

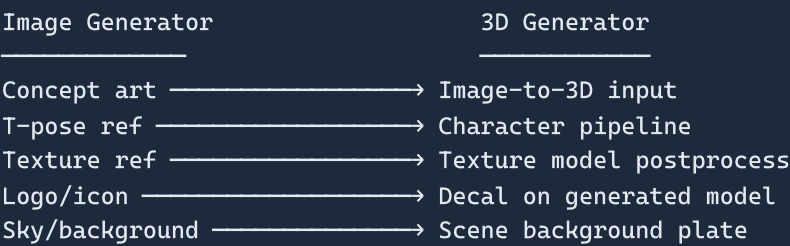
纹理参考:

Create a seamless game texture reference for [surface].
Orthographic/top-down, PBR-friendly albedo, clear material variation,
no perspective, no baked strong shadows, [style/material details].

4.8.5 文件组织

```
assets/concepts/    ← Concepts + image-to-3D sources
assets/textures/    ← Textures + decals
assets/decals/
assets/ui/          ← Icons + GUI source images
```

4.8.6 与 3D Generator 的协作



4.9 threejs-audio-generator（音频生成器） — ElevenLabs API 集成

职责: 游戏音频全流程 —— SFX、环境音、语音、转换、清理。

位置: `skills/threejs-audio-generator/`

4.9.1 内部结构

```
threejs-audio-generator/
├── SKILL.md
├── scripts/
│   └── threejs_audio_asset.py          # ElevenLabs API 客户端
├── references/
│   └── audio-workflows.md             # 音频 workflow
```

4.9.2 音频参数默认值

类型	时长	Prompt Influence	格式	备注
SFX	0.5-2.5s	0.55-0.8	mp3_44100_128	瞬态清晰
UI 音效	0.15-0.8s	高	mp3_44100_128	短促明确

类型	时长	Prompt Influence	格式	备注
环境循环	8-30s	0.3-0.55	mp3_44100_128	--loop flag
TTS 语音	自动	N/A	mp3_44100_128	--voice-id
语音转换	同输入	N/A	同输出	voice-change + 可选降噪

4.9.3 游戏音频钩子映射

```
UI click/pause/retry ———> SFX (0.2s, high influence)
Pickup/score —————> SFX (0.5s, bright transient)
Damage/fail —————> SFX (1.0s, impact character)
Boost/speed —————> SFX (1.5s, rising pitch)
Combo/milestone —————> SFX (1.0s, escalating)
Ambience —————> SFX with --loop (12s, low influence)
Announcement —————> TTS (--voice-id, --text)
Boss dialogue —————> voice-change (--input scratch.wav)
```

4.9.4 Three.js Web Audio 集成

```
// AudioContext resume (需用户手势)
document.addEventListener('click', () => {
  audioContext.resume();
}, { once: true });

// 加载 + 播放 SFX
const buffer = await fetch('assets/audio/sfx/pickup.mp3')
  .then(r => r.arrayBuffer())
  .then(b => audioContext.decodeAudioData(b));

function playSfx(buffer: AudioBuffer) {
  const source = audioContext.createBufferSource();
  source.buffer = buffer;
  source.connect(masterGain);
  source.start();
}

// 环境循环
const ambienceSource = audioContext.createBufferSource();
ambienceSource.buffer = ambienceBuffer;
ambienceSource.loop = true;
ambienceSource.connect(ambienceGain);
ambienceSource.start();

// 暂停/恢复
ambienceGain.gain.value = 0; // mute
ambienceSource.stop();      // stop on scene teardown
```

5. 工作原理解析

5.1 从用户输入到最终交付的完整流程

用户: "Use threejs-game-director to build a premium futuristic hover racing game from scratch."

↓

Step 1: Director 加载自身 SKILL.md + director-phase-os.md

Step 2: 尝试加载 8 个 sibling SKILL.md 文件

Path: ../<skill>/SKILL.md → ~/.claude/ → ~/.codex/ → skills/

必装: gameplay, aaa-graphics, ui, debug, qa (全部)

条件: 3d-gen, image-gen, audio-gen (premium hover racing)

Step 3: Phase 1 - Discovery

→ 项目不存在 → 创建脚手架

python3 ... /create_threejs_game.py ./hover-racing

→ cd hover-racing && npm install && npm run dev

→ 定义玩法: "Pilot hover vehicle through neon city, dodge obstacles, collect energy orbs, beat time target"

Step 4: Phase 2 - Gameplay

→ 加载 gameplay-workflows.md + physics-engine-selection.md

→ 创建: 赛道/城市环境, 玩家载具, 障碍物, 收集物, 碰撞, HUD

→ 物理: Rapier (hover physics + ramp collisions + CCD)

→ 调校: 速度, 加速, 相机跟随, 漂移手感

→ 验证: build → browser → screenshot → canvas pixel → input

Step 5: Phase 3 - External Asset Sourcing

→ 运行 probe_asset_credentials.sh

输出: TRIPO_API_KEY=SET, GEMINI_API_KEY=SET, ELEVENLABS=SET

→ 加载 3d-gen SKILL.md, image-gen SKILL.md, audio-gen SKILL.md

→ 决策:

Hero hover vehicle → Tripo image-to-3D (from Gemini concept)

City buildings → procedural WorldPropKit

Energy orbs → procedural + Gemini decal icons

Sky/nebula → Gemini image generator

SFX (boost/pickup) → ElevenLabs sfx commands

Ambience loop → ElevenLabs sfx --loop

→ 生成: hero-vehicle.glb, sky-nebula.png, sfx/*.mp3

Step 6: Phase 4 - AAA Graphics

→ 加载 visual-scorecard.md + implementation-blueprint.md
+ model-recipes.md + render-recipes.md

→ 初始评分: 大部分 0-1

→ 建立: MaterialLibrary, WorldPropKit, VfxSystem, LightingRig

→ 导入 hero-vehicle.glb (GLTFLoader + AnimationMixer)

→ 构建赛道套件, 城市中远景, 霓虹标志

→ 灯光栈: 紫色/青色赛博朋克风格

→ VFX: boost trails, pickup rings, hit sparks, speed lines

→ 迭代评分至每类 ≥ 2, 平均 ≥ 2.3

Step 7: Phase 5 – UI → 加载 ui-patterns.md + checklists → HUD: 速度计 + 能量条 + 计时器 + 圈数 (非 stat cards) → Pause overlay + Fail/Retry modal → Mobile: 虚拟摇杆 + 加速按钮 + 安全区 → 验证: 桌面+移动截图, 文字适配, 触控目标	
Step 8: Phase 6 – Debug & Profile → 加载 debug-profile-checklists.md → 性能基线: draw calls, triangles, textures, bundle size → InstancedMesh 用于建筑窗户/赛道指示灯 → LOD 用于背景建筑 → DPR cap = 2 → 优化后重新测量	
Step 9: Phase 7 – QA & Release → 加载 qa-release-checklists.md → npm run build → production preview → Playwright 截图 (desktop + mobile) → inspect-threejs-canvas.mjs → nonblank ✓ → Console errors ✓, page errors ✓ → Main loop: move → collect → dodge → win → retry → Production: base path, debug gating, API key scan → 部署到 Netlify (或类似 static host)	
Step 10: Final Report → 草拟报告 → audit_reference_report.py --premium --physics → 包含: 四大账本 + 视觉评分卡 + 截图 + 诊断 + 风险 → 交付: "Premium hover racing game ready"	

5.2 质量门禁触发条件

请求关键词	触发行为
"build", "create", "from scratch"	加载所有 5 核心 + 使用脚手架
"premium", "AAA", "showcase"	+ 条件加载 3 生成器 + 视觉评分卡 ≥ 2.3
"high-fidelity", "less basic"	+ 条件加载 3 生成器
"polish", "upgrade", "finish"	加载所有 5 核心 + 视觉评分卡
"debug", "fix", "broken"	加载 debug-profiler + 场景调试清单
"release", "deploy", "ship"	加载 qa-release + release checklist
"physics", "collision", "ragdoll"	+ physics-engine-selection.md

6. 总结与评估

6.1 核心创新点

创新	说明
多层门禁系统	技能加载 → 参考文件 → 外部资产 → 评分卡 → 审计器，每层不可跳过
视觉评分卡量化	10 类别 0-3 分标准化，消除 "看起来不错" 的主观判断
凭据安全探测	<code>probe_asset_credentials.sh</code> 以登录 shell 方式 source profile，只输出 SET/MISSING，永不泄露密钥值
四大账本追踪	Skill-loading + Reference + Asset Sourcing + Phase Execution 全流程可审计
自包含技能包	每个技能含 SKILL.md + references + scripts + assets，安装即用
报告自动审计	<code>audit_reference_report.py</code> 正则匹配验证，防止声明 premium 但缺失评分卡/资产证据
脚手架内建诊断	<code>window.__THREE_GAME_DIAGNOSTICS__</code> 实时暴露渲染器+游戏状态
混合资产管线	Procedural + AI-generated + Hybrid，按 surface 价值而非全量生成
物理引擎选择方法论	基于游戏类型的阶梯式决策树，非 "用 Rapier 就行"

6.2 技术局限

局限	影响
Claude 兼容性	Claude 的 skill runner 可能不支持动态调用其他技能 → fallback 到硬编码的 phase-os.md
无代码生成验证	依赖 AI 生成代码的质量，无语法/语义级自动纠错
外部 API 依赖	3D/Image/Audio 生成依赖付费 API (Tripo、Gemini、ElevenLabs)
浏览器测试限制	Playwright 截图+像素采样不能完全替代人类游戏测试
无版本控制集成	资产生成账本记录的 task ID 和文件路径无自动版本追踪
评分卡主观性	0-3 分的评分仍依赖 AI 对视觉质量的判断，可能存在幻觉
移动端覆盖	主要依赖 Playwright 移动视口模拟，非真机测试

6.3 适用场景

✔ 适合:

- 快速原型 → Premium 品质的完整游戏开发
- AI 辅助游戏开发的标准化流程
- 对质量有要求的 Three.js 浏览器游戏项目
- 需要 AI 自动生成 3D/2D/音频资产的场景
- 团队中缺少专业美术/音频人员的项目

✘ 不适合:

- 非 Three.js 的游戏引擎 (Unity/Unreal/Godot)
- 需要自定义物理引擎的复杂模拟
- 多人/网络游戏 (系统未涵盖网络层)
- 对资产生成结果有精确要求的商业化项目 (AI 生成不可精确控制)
- 简单的静态 3D 场景展示 (门禁开销过大)

6.4 架构评分

维度	评分	说明
完整性	★★★★★	覆盖从脚手架到发布的完整游戏开发生命周期
一致性	★★★★★	所有技能遵循统一的 SKILL.md + references + scripts 结构
可维护性	★★★★☆	自包含包结构便于独立更新，但 director 和 phase-os.md 有知识重复
可扩展性	★★★★☆	新技能遵循统一规范即可接入，但门禁规则需手动更新
文档质量	★★★★★	SKILL.md + references + checklists 三层文档体系非常完善
安全性	★★★★☆	凭据探测不泄露密钥，但 API key 传播风险依赖 AI 知晓该规范

结论: threejs-game-skills 是一套精心设计的、将 AAA 级浏览器游戏开发知识编码为 AI 可执行指令的知识系统。其核心价值在于：通过严谨的多层门禁、量化评分卡和自动化审计，将 "做一个能跑的游戏" 和 "做一个 Premium/AAA 品质的游戏" 之间的巨大鸿沟系统性地桥接起来。